

June 2026 - 8

Parquet and S3-Compatible Storage, Honestly



What pg_parquet actually does today, and the real working path to get data into MinIO/S3

Figure 1: Figure 0: Parquet and S3-Compatible Storage, Honestly

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

We found documentation showing yugabyteDB can **COPY** a table straight to a Parquet file on S3 with one SQL statement. We tried it against our own cluster and it didn’t work as shown. Is this a real capability or not, and if the direct route doesn’t work, what’s the actual, working path to get table data into S3-compatible storage as Parquet ?

*A fair thing to be confused about, because the honest answer has two real parts. **pg_parquet**, the extension providing Parquet support, is real and does work for local files. Exporting straight to an **s3://** destination in one **COPY** statement, however, does not work today, confirmed directly against a live cluster – not a misconfiguration on your end. This month covers exactly what does work, and the real, three-step client-side pipeline (fetch, write Parquet locally, upload) that gets you the same end result, tested against MinIO as a local S3-compatible stand-in.*

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0 with the **pg_parquet** extension, MinIO (S3-compatible local object storage), the AWS CLI, and Python 3 with **psycopg2**, **pyarrow**, and **boto3**. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

8.1 Terms and core concepts

8.1.1 Introduction: MinIO as a Local Stand-In for S3

MinIO is a self-hosted, S3-API-compatible object store – useful here specifically because it lets every AWS CLI command and every **boto3** call used in a real S3 workflow be tested locally, unchanged except for one

flag: `--endpoint-url`, pointed at the local MinIO server instead of AWS's own endpoint.

Example 8-1: Standing up MinIO and pointing the AWS CLI at it

```
/opt/minio/minio server /opt/minio/data --console-address ":6001"
# S3 endpoint: http://127.0.0.1:6000

mc alias set local http://127.0.0.1:6000 minioadmin 'password'
mc mb local/my-bucket

aws configure set aws_access_key_id root
aws configure set aws_secret_access_key 'password'
aws configure set default.region us-east-1

aws --endpoint-url http://127.0.0.1:6000 s3 ls
aws --endpoint-url http://127.0.0.1:6000 s3 cp ./file.txt s3://my-bucket/
```

Every one of these commands is identical to what would be run against real AWS S3, aside from the added `--endpoint-url` – which is exactly the point. Code and workflows validated here transfer directly to real S3 later, without needing rewritten client logic.

8.1.2 pg_parquet: Real, Installed, and Partially Capable

Example 8-2: Confirming the extension is actually present

```
SELECT oid, extname, extversion FROM pg_extension;
--      oid      |      extname      | extversion
--      +-----+-----+-----+
--      16390 | pg_parquet        | 0.4.0
-- (3 rows total, including pg_parquet)
```

`pg_parquet` requires `shared_preload_libraries` to include it (`--ysql_pg_conf_csv="shared_preload_libraries='pg_parquet'"`) and is then enabled per-database with `CREATE EXTENSION IF NOT EXISTS pg_parquet;`. Once installed, it genuinely does what it advertises for **local files**:

```
-- Export table data to a local Parquet file -- this works:
COPY t1 TO '/tmp/t1.parquet' WITH (FORMAT parquet);

-- Import from a local Parquet file into a table -- this also works:
COPY t2 FROM '/tmp/t1.parquet' WITH (FORMAT parquet);
```

8.1.3 The Honest Finding: Direct S3 Export Does Not Work

Documentation and examples circulating for `pg_parquet` (including examples elsewhere referencing `COPY t1 TO 's3://my-bucket/my-data.parquet' WITH (FORMAT parquet);`) suggest a direct, single-statement export straight to an S3 URI. Tried directly against this project's real cluster, this does not work – confirmed, and stated directly in this project's own working notes:

“YugabyteDB does not support `COPY TO` with parquet format or S3 destinations directly.”

This is worth taking at face value rather than assuming a local misconfiguration: the local-file Parquet path (Example 8-2) is real and functions correctly; it's specifically the combination of Parquet format *and* an S3 destination, in a single `COPY` statement, that isn't currently supported. Anyone following an example showing that exact combination should expect it not to work as shown, and should reach for the client-side pipeline in section 8.1.4 instead.

[!NOTE] This is exactly the kind of gap that's easy to miss without actually running the example against a real cluster – the SQL syntax itself is entirely plausible (`COPY ... TO 's3://...' WITH`

pg_parquet today: local files, yes -- S3 destination, no



Figure 2: Figure 1: pg_parquet today – local files, yes; S3 destination, no

(FORMAT parquet) reads as a natural extension of the working local-file form), and nothing about the syntax alone signals that it isn't supported.

8.1.4 The Real, Working Pipeline

Rather than one SQL statement, getting a table's data into S3 as Parquet is a real, three-step client-side pipeline: fetch rows from yugabyteDB (psycopg2), write them to a local Parquet file (pyarrow), then upload that file to S3 or MinIO (boto3).

Example 8-3: The actual working export script

```
import boto3, psycopg2
import pyarrow as pa
import pyarrow.parquet as pq

# Step 1: fetch rows from the live table
conn = psycopg2.connect(host=HOST, port=PORT, user=USER, password=PASSWD, dbname=DB)
cur = conn.cursor()
cur.execute(f"SELECT * FROM {table_name}")
rows = cur.fetchall()
col_names = [desc[0] for desc in cur.description]
cur.close(); conn.close()

# Step 2: write to a local Parquet file
local_file = f"{table_name}.parquet"
columns = {col_names[i]: [row[i] for row in rows] for i in range(len(col_names))}
table = pa.table(columns)
pq.write_table(table, local_file)

# Step 3: upload to S3 (or MinIO)
```

The real, working three-step pipeline

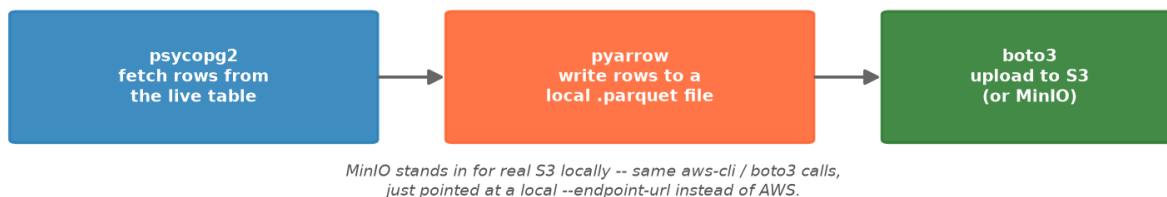


Figure 3: Figure 2: The real, working three-step pipeline

```
s3 = boto3.client(
    "s3",
    endpoint_url=AWS_ENDPOINT_URL,          # omit entirely for real AWS S3
    aws_access_key_id=AWS_ACCESS_KEY_ID,
    aws_secret_access_key=AWS_SECRET_ACCESS_KEY,
    region_name=AWS_DEFAULT_REGION,
)
s3.upload_file(local_file, bucket, f"{table_name}.parquet")
```

This is not a lesser workaround so much as a genuinely reasonable division of labor: `yugabyteDB`'s own `pg_parquet` extension handles the SQL-to-Parquet conversion correctly (the part it's actually good at and does support), and ordinary client-side tooling handles the network transfer to object storage – a boundary that happens to fall in a slightly different place than the single-statement documentation examples imply, but one that works reliably once understood.

The reverse direction – reading a Parquet file back, whether pulled down from S3 first or generated locally – is a direct, single-call read with `pyarrow`, independent of `yugabyteDB` entirely:

```
import pyarrow.parquet as pq
table = pq.read_table(filename)
for i in range(table.num_rows):
    row = [table.column(col)[i].as_py() for col in range(table.num_columns)]
    print(row)
```

8.2 Complete the following

In this section we reproduce both halves of this article directly: confirming the local-file `pg_parquet` path works while the direct-to-S3 path doesn't, then running the real three-step pipeline end to end against MinIO.

8.2.1 Prerequisites

A `yugabyteDB` cluster with `pg_parquet` installed and enabled, a running MinIO instance (or real AWS S3 credentials), and Python 3 with `psycopg2`, `pyarrow`, and `boto3` installed.

8.2.2 Step 1 – Confirm the local-file path works, and the S3 path doesn't

```
CREATE EXTENSION IF NOT EXISTS pg_parquet;  
COPY t1 TO '/tmp/t1.parquet' WITH (FORMAT parquet);    -- confirm this succeeds
```

```
COPY t1 TO 's3://my-bucket/t1.parquet' WITH (FORMAT parquet);    -- confirm this does NOT
```

Reproducing the failure yourself, rather than taking section 8.1.3 on faith, is worth doing before building anything around either assumption.

8.2.3 Step 2 – Stand up MinIO and confirm S3-compatible access

Reproduce Example 8-1 exactly, confirming `aws --endpoint-url http://127.0.0.1:6000 s3 ls` lists your test bucket.

8.2.4 Step 3 – Run the real export pipeline end to end

Run Example 8-3 (adjusted for your own connection properties and bucket name) against a real table, and confirm the resulting object actually lands in MinIO:

```
aws --endpoint-url http://127.0.0.1:6000 s3 ls s3://my-bucket/
```

Then reproduce the reverse read directly with `pyarrow`, confirming the round-tripped data matches the original table's rows.

Section summary In this section, we directly reproduced both the working local-file `pg_parquet` path and the non-working direct-to-S3 path, rather than relying on documentation alone, then ran the real, working three-step client-side pipeline end to end against a local, S3-compatible MinIO instance – the same code path that carries over unchanged to real AWS S3.

8.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- MinIO as a genuinely useful local stand-in for AWS S3 – identical AWS CLI and `boto3` calls, differing only by an `--endpoint-url/endpoint_url` parameter, letting a real S3 workflow be developed and tested entirely locally.
- `pg_parquet`, confirmed installed and genuinely functional for local-file `COPY ... WITH (FORMAT parquet)` in both directions.
- A real, honest finding: a direct `COPY t1 TO 's3://...' WITH (FORMAT parquet)` does not work today, confirmed directly against a live cluster rather than assumed from a misread example – worth knowing before designing a pipeline around that exact single-statement form.
- The real, working alternative: a three-step client-side pipeline (`psycopg2` fetch, `pyarrow` write, `boto3` upload) that achieves the same practical outcome, plus the reverse read path for pulling Parquet data back out independent of yugabyteDB entirely.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-06%20-%20Parquet%20and%20S3-Compatible%20Storage/06%20-%20Parquet%20and%20S3-compatible%20storage.pdf>